

Carnet de Bord

Robot Auto-Équilibré STM32F4

Développement d'un robot auto-équilibré capable de se déplacer et d'évoluer vers la navigation autonome.

Principe du projet

Pourvu qu'aujourd'hui soit mieux qu'hier.

Progresser chaque jour, même légèrement, est une victoire.

Auteur :	Ithiel Gabriel Djifa DENYIGBA
Projet :	Robot auto-équilibré STM32F4
Version :	1.0
Début :	Juillet 2026
Budget initial :	200 €

Table des matières

1	Pourquoi ce projet ?	2
2	Cahier des charges	2
3	Roadmap de développement	3
4	Journal de bord	4
5	Les composants	13
6	Notions théoriques	14
7	Architecture du robot	15
8	Les erreurs et leçons apprises	15
9	Améliorations futures	16
10	Matériel et plan d'achat	16
	Conclusion	17

1. Pourquoi ce projet ?

Ce projet est un laboratoire personnel pour apprendre et mettre en pratique les notions fondamentales des systèmes embarqués, de l'automatique et de la robotique mobile.

Il doit permettre de développer des compétences solides en :

- systèmes embarqués ;
- programmation STM32 ;
- électronique appliquée ;
- contrôle moteur ;
- automatique et PID ;
- traitement du signal ;
- fusion de capteurs ;
- robotique mobile ;
- architecture logicielle ;
- gestion de projet technique.

L'objectif n'est pas seulement de construire un robot. Le robot est le résultat visible. Le véritable objectif est de progresser comme ingénieur en apprenant à concevoir, tester, corriger, documenter et améliorer un système réel.

Principe du projet

Le robot n'est pas seulement une machine. Il devient un support d'apprentissage, un terrain d'expérimentation et une preuve concrète de progression.

Pourvu qu'aujourd'hui soit mieux qu'hier.

2. Cahier des charges

Objectif général

Construire un robot auto-équilibré basé sur une carte STM32F4, capable de se stabiliser, de se déplacer et, à terme, d'évoluer vers une navigation autonome.

Version 1 – Stabilisation de base

Le robot doit :

- tenir debout en équilibre ;
- avancer ;
- reculer ;
- être contrôlé par une boucle temps réel ;
- exploiter une IMU pour estimer son inclinaison ;
- utiliser des moteurs avec encodeurs pour mesurer le mouvement.

Version 2 – Robot connecté

Ajouter :

- Wi-Fi via ESP32 ;
- télémétrie vers PC ;
- réglage des paramètres PID à distance ;
- contrôle manuel depuis une interface distante.

Version 3 – Navigation autonome

Ajouter :

- navigation d'un point A vers un point B ;
- estimation de déplacement par odométrie ;
- évitement d'obstacles ;
- planification de trajectoire simple.

Contraintes de conception

- privilégier l'apprentissage progressif ;
- valider chaque brique séparément avant intégration ;
- garder une architecture logicielle claire ;
- documenter chaque séance ;
- éviter d'ajouter des fonctionnalités avancées avant stabilisation.

3. Roadmap de développement

Phase	Objectif	Compétences travaillées
Phase 1	Prise en main STM32	GPIO, UART, Timers, PWM, interruptions, ADC
Phase 2	Briques robot	IMU, I2C, moteur, encodeur, filtre complémentaire
Phase 3	Intégration mécanique	châssis, câblage, alimentation, centre de gravité
Phase 4	Stabilisation	PID angle, boucle temps réel, tuning
Phase 5	Déplacement	vitesse, odométrie, contrôle dynamique
Phase 6	Autonomie	Wi-Fi, télémétrie, navigation, évitement d'obstacles

Principe du projet

Chaque phase est une marche. L'objectif n'est pas de sauter les étapes, mais de les valider proprement.

4. Journal de bord

Chaque séance suit ce format :

- Objectif ;
- Réalisation ;
- Problèmes rencontrés ;
- Solution trouvée ;
- Apprentissage ;
- Prochaine étape.

Séance 0 – Naissance du projet

Journal de séance

Date : Juillet 2026

Objectif :

Définir la vision du projet, concevoir son architecture et sélectionner l'ensemble du matériel nécessaire à la réalisation du robot auto-équilibré.

Réalisation :

- Définition de la vision du projet.
- Création du carnet de bord.
- Élaboration de la feuille de route de développement.
- Définition des différentes phases du projet.
- Étude des architectures possibles.
- Comparaison des différentes cartes STM32.
- Sélection des moteurs, capteurs et composants.
- Vérification de la compatibilité de tous les éléments.
- Validation de l'architecture électrique.
- Commande de l'ensemble des composants.

Matériel commandé :

- STM32 Nucleo-F446RE.
- Module IMU MPU6050.
- Driver moteur TB6612FNG.
- Deux motoréducteurs avec encodeurs.
- Batterie LiPo 2S 7,4 V – 2200 mAh.
- Chargeur LiPo avec équilibrage.
- Convertisseur Buck LM2596.
- Connecteurs Deans avec câbles silicone.
- Interrupteur ON/OFF.
- Multimètre numérique.
- Cutter de précision.
- Plaques de PVC expansé.
- Fils de câblage.
- Câbles Dupont.

Budget du projet :

200 €

Version 1

Difficultés rencontrées :

- Choisir entre plusieurs cartes STM32.
- Comprendre l'alimentation d'un robot mobile.
- Sélectionner une batterie compatible.
- Comprendre le rôle du Buck Converter.
- Choisir des moteurs adaptés au contrôle d'équilibre.
- Vérifier la compatibilité entre tous les composants.

Ce que j'ai appris :

Avant de programmer un robot, il faut d'abord le concevoir. Un bon projet commence par une architecture solide. Le choix des composants est aussi important que les algorithmes qui les feront fonctionner.

Prochaine séance :

Installer STM32CubeIDE et commencer la prise en main de la STM32 avec le premier programme GPIO.

Principe du projet

Aujourd'hui, je n'ai encore écrit aucune ligne de code.

Pourtant, le projet est déjà né.

Chaque grande réalisation commence par une décision.

Aujourd'hui, j'ai décidé de construire quelque chose qui me fera progresser.

Pourvu qu'aujourd'hui soit mieux qu'hier.

Séance 1 – GPIO

Journal de séance

Date : Juillet 2026

Objectif :

Faire clignoter la LED intégrée de la carte STM32 Nucleo-F446RE afin de valider la première interaction réelle avec le microcontrôleur.

Objectif pédagogique :

Comprendre le principe d'un GPIO non pas comme une simple fonction logicielle, mais comme un périphérique matériel contrôlé par des registres mémoire. L'objectif était aussi de commencer à comprendre la relation entre le processeur, la carte mémoire, les bus internes, les registres et les broches physiques du microcontrôleur.

Réalisation :

- Mise en place du dossier de travail de la séance 1.
- Création d'un projet STM32 minimal nommé `allume_led`.
- Mise en place d'un workflow de développement basé sur VS Code, CMake, Ninja, la toolchain `arm-none-eabi-gcc` et OpenOCD.
- Vérification de la détection du ST-Link intégré de la carte Nucleo-F446RE sous Linux.
- Compilation réussie du projet en mode `Debug`.
- Écriture d'un programme de clignotement de LED sans HAL, par accès direct aux registres.
- Flash du programme sur la STM32 avec OpenOCD.
- Validation expérimentale : la LED LD2 reliée à PA5 clignote correctement.
- Mise en place d'un fichier `.gitignore` propre pour éviter de versionner les fichiers générés.
- Commit Git et synchronisation du projet avec GitHub.

Choix technique :

Pour cette séance, le choix a été fait de ne pas utiliser HAL. L'objectif n'était pas seulement d'obtenir un résultat fonctionnel, mais de comprendre le fonctionnement interne du microcontrôleur. Le programme a donc été écrit en manipulant directement les adresses mémoire et les registres nécessaires.

Problèmes rencontrés :

- Le bouton *Launch STM32CubeMX* de l'extension VS Code ne trouvait pas STM32CubeMX, car STM32CubeMX n'était pas installé ou pas présent dans le chemin système.
- Le projet créé était un projet minimal et non un projet STM32Cube/HAL.
- La première configuration CMake échouait car Ninja n'était pas installé.
- La compilation échouait ensuite car la toolchain ARM `arm-none-eabi-gcc` n'était pas disponible dans le PATH.
- Certaines fonctions graphiques avancées de l'extension STM32 pour VS Code n'étaient pas directement disponibles car le projet n'était pas reconnu comme un projet STM32Cube complet.

Solutions trouvées :

- Utilisation du terminal comme workflow principal pour compiler et flasher le programme.
- Installation de Ninja, CMake et de la toolchain ARM.
- Compilation avec `cmake -preset Debug` puis `cmake -build -preset Debug`.
- Flash de la carte avec OpenOCD via l'interface ST-Link.
- Décision de conserver ce premier projet comme projet pédagogique minimal, centré sur la compréhension des registres et du fonctionnement matériel.

Notions comprises :

- Un GPIO est un périphérique matériel interne au microcontrôleur.
- Le processeur ne commande pas directement une LED : il écrit dans des registres mémoire.
- La STM32 utilise le principe du *Memory-Mapped I/O* : les périphériques sont accessibles à travers des adresses mémoire.
- `GPIOA_BASE` représente l'adresse de base du périphérique GPIOA.
- Les registres d'un périphérique sont situés à des offsets fixes à partir de son adresse de base.
- Le registre `MODER` configure le mode des broches GPIO.
- Le registre `ODR` permet de modifier l'état logique des sorties.
- Avant d'utiliser GPIOA, il faut activer son horloge via le registre `RCC_AHB1ENR`.
- AHB signifie *Advanced High-performance Bus*. C'est un bus interne rapide reliant le processeur à certains périphériques.
- Le mot-clé `volatile` est essentiel pour empêcher le compilateur d'optimiser les accès aux registres matériels.
- Les opérations binaires comme le décalage, le masquage et le XOR permettent de modifier précisément certains bits d'un registre.

Explication synthétique du fonctionnement :

Le processeur Cortex-M4 écrit dans le registre d'activation d'horloge du RCC afin d'activer le périphérique GPIOA. Ensuite, il configure la broche PA5 en sortie en modifiant les deux bits correspondants dans le registre `MODER`. Une fois PA5 configurée en sortie, le programme inverse régulièrement le bit 5 du registre `ODR`. Cette modification logique se traduit matériellement par un changement de tension sur la broche PA5, ce qui provoque l'allumage puis l'extinction de la LED LD2.

Méthode d'apprentissage retenue :

La méthode retenue pour la suite du projet est de comprendre chaque périphérique avant de l'utiliser. Pour chaque nouvelle brique, il faudra identifier le rôle du périphérique, sa position dans la carte mémoire, ses registres principaux, puis écrire le code correspondant. Le but n'est pas seulement d'utiliser une bibliothèque, mais d'apprendre à lire la documentation constructeur et à raisonner directement sur l'architecture du microcontrôleur.

Résultat :

La LED LD2 de la carte Nucleo-F446RE clignote correctement. La chaîne complète de développement est validée : édition du code, compilation, génération de l'exécutable, flash avec ST-Link et exécution sur la carte.

Apprentissage principal :

Le processeur ne commande jamais directement une LED. Il écrit dans des registres mémoire, et ce sont les périphériques matériels qui transforment ces écritures en actions physiques.

Prochaine étape :

Commencer la séance 2 avec l'UART afin de mettre en place un premier outil de debug série. Cette étape permettra d'afficher des messages depuis la STM32 vers le PC et de mieux observer l'état interne du programme pendant les essais.

Séance 2 – UART

Journal de séance

Objectif : Mettre en place un système de debug via terminal série.

Réalisation :

Configurer l'UART de la STM32 et afficher des messages sur un terminal PC.

Résultat attendu :

Afficher un message de type `Hello Robot` dans le terminal.

Apprentissage :

La communication série devient l'outil principal de diagnostic pendant tout le projet.

Séance 3 – IMU : première communication

Journal de séance

Objectif : Vérifier que le module MPU6050 communique correctement avec la STM32.

Technologie : I2C.

Réalisation :

Lire le registre d'identification de l'IMU et afficher la réponse via UART.

Résultat attendu :

Confirmer que le capteur répond à l'adresse I2C prévue.

Apprentissage :

Avant d'exploiter un capteur, il faut d'abord valider sa communication électrique et logicielle.

Séance 4 – PWM moteur

Journal de séance

Objectif : Générer un signal PWM pour contrôler la vitesse moteur.

Technologie : Timers + PWM.

Réalisation :

Configurer un timer STM32 en mode PWM et faire varier le rapport cyclique.

Apprentissage :

Le PWM permet de contrôler l'énergie moyenne envoyée au moteur.

Séance 5 – Encodeurs

Journal de séance

Objectif : Mesurer la vitesse réelle des moteurs.

Technologie : Timer en mode encodeur.

Réalisation :

Connecter les voies A et B d'un encodeur à deux entrées timer et lire la position moteur.

Apprentissage :

L'encodeur permet de passer d'une commande aveugle à une mesure réelle du mouvement.

Séance 6 – Lecture moteur + sens de rotation

Journal de séance

Objectif : Contrôler la direction et le sens des moteurs.

Technologie : GPIO + PWM + Driver moteur TB6612FNG.

Réalisation :

- inversion du sens de rotation ;
- test avant/arrière ;
- validation du pont en H TB6612FNG.

Problème possible :

Le moteur tourne dans le mauvais sens.

Solution :

Inversion des broches de direction IN1/IN2 ou correction logicielle du signe de commande.

Apprentissage :

Un moteur ne se contrôle pas seulement en vitesse, mais aussi en direction.

Séance 7 – Lecture IMU brute

Journal de séance

Objectif : Lire les données brutes de l'IMU.

Technologie : I2C + MPU6050.

Réalisation :

- lecture des registres d'accélération ;
- lecture du gyroscope ;
- affichage UART.

Problème attendu :

Valeurs instables et bruitées.

Apprentissage :

Les capteurs bruts ne sont jamais directement exploitables pour le contrôle.

Séance 8 – Filtre complémentaire

Journal de séance

Objectif : Obtenir un angle stable.

Technologie : Fusion accéléromètre + gyroscope.

Réalisation :

- implémentation du filtre complémentaire ;
- estimation de l'angle de tangage ;
- stabilisation des mesures.

Problème attendu :

Dérive gyroscopique et bruit de l'accéléromètre.

Solution :

Pondérer l'information du gyroscope et de l'accéléromètre.

Apprentissage :

La fusion de capteurs est une base des systèmes autonomes.

Séance 9 – Test moteur en boucle ouverte

Journal de séance

Objectif : Tester les moteurs sans contrôle fermé.

Technologie : PWM fixe.

Réalisation :

- test à vitesse constante ;
- test sous différentes charges ;
- observation du comportement mécanique.

Problème attendu :

La vitesse varie selon la charge et l'état de la batterie.

Apprentissage :

La boucle ouverte est insuffisante pour un système dynamique précis.

Séance 10 – Boucle de contrôle vitesse

Journal de séance

Objectif : Stabiliser la vitesse moteur.

Technologie : PID vitesse + encodeurs.

Réalisation :

- implémentation d'un PID de vitesse ;
- lecture encodeur ;
- ajustement automatique du PWM.

Problème attendu :

Oscillations ou réponse trop lente.

Solution :

Régler progressivement les gains, en commençant par le gain proportionnel.

Apprentissage :

Un PID mal réglé crée de l'instabilité au lieu de la corriger.

Séance 11 – Boucle d'équilibre en simulation

Journal de séance

Objectif : Tester le contrôle d'équilibre avant l'intégration physique.

Technologie : Simulation logicielle d'un pendule inversé simplifié.

Réalisation :

- modélisation simplifiée de l'angle ;
- test du PID angle ;
- validation du comportement attendu.

Apprentissage :

Toujours tester un contrôle avant de l'appliquer directement à un système instable réel.

Séance 12 – Construction du châssis

Journal de séance

Objectif : Construire la base mécanique du robot.

Technologie : Mécanique + intégration électronique.

Réalisation :

- assemblage de la structure du robot ;
- fixation des moteurs ;
- positionnement de la batterie pour maîtriser le centre de gravité ;
- installation de la STM32 et du driver moteur ;

- câblage initial.

Points critiques :

- équilibre mécanique ;
- rigidité du châssis ;
- alignement des roues ;
- fixation stable de l'IMU.

Apprentissage :

La stabilité d'un robot commence en mécanique, pas en code.

Séance 13 – Première tentative d'équilibre

Journal de séance

Objectif : Faire tenir le robot debout.

Technologie : IMU + PID angle + PWM.

Réalisation :

- boucle de contrôle à 500–1000 Hz ;
- lecture IMU en temps réel ;
- correction moteur selon l'angle.

Résultat probable :

Oscillations rapides et instabilité initiale.

Apprentissage :

Un système instable doit être réglé, pas abandonné.

Séance 14 – Stabilisation progressive

Journal de séance

Objectif : Améliorer la stabilité du robot.

Actions :

- réglage progressif du PID ;
- réduction du bruit IMU ;
- amélioration de la fréquence de boucle ;
- observation du comportement mécanique.

Résultat attendu :

Réduction des oscillations et maintien de l'équilibre pendant plusieurs secondes.

Apprentissage :

Le contrôle est un processus itératif.

Séance 15 – Déplacement contrôlé

Journal de séance

Objectif : Faire avancer et reculer le robot sans tomber.

Technologie : Contrôle angle + consigne vitesse.

Réalisation :

- ajout d'une consigne de vitesse ;
- couplage entre équilibre et déplacement ;
- test de stabilité dynamique.

Apprentissage :

Stabilité et mouvement forment un système couplé.

Séance 16 – Préparation autonomie

Journal de séance

Objectif : Préparer une navigation simple d'un point A vers un point B.

Technologie : Odométrie + contrôle position.

Réalisation :

- estimation de la distance parcourue ;
- correction de trajectoire ;
- test d'un mouvement simple sur une distance donnée.

Apprentissage :

Un robot stable est une base pour l'autonomie.

5. Les composants

STM32 Nucleo-F446RE

Fiche composant

- Microcontrôleur STM32F446RE.
- Coeur ARM Cortex-M4.
- Timers adaptés au PWM et aux encodeurs.
- Interfaces UART, SPI, I2C.
- ADC, DMA et interruptions.
- ST-Link intégré pour programmation et debug.

IMU MPU6050

Fiche composant

- Accéléromètre 3 axes.
- Gyroscope 3 axes.
- Communication I2C.
- Utilisé pour estimer l'angle du robot.

Moteurs avec encodeurs

Fiche composant

- Moteurs DC avec réduction mécanique.
- Encodeurs quadrature voies A et B.
- Mesure de vitesse et de position.
- Utilisés pour le contrôle de vitesse et l'odométrie.

Driver moteur TB6612FNG

Fiche composant

- Double pont en H.
- Contrôle de deux moteurs DC.
- Commande par GPIO et PWM.
- Alimentation moteur séparée de la logique.

Alimentation

Fiche composant

- Batterie LiPo 2S 7,4 V.
- Convertisseur Buck LM2596 pour obtenir une tension logique stable.
- Interrupteur ON/OFF sur la ligne positive.
- Connecteurs Deans pour la puissance.

6. Notions théoriques

GPIO

Les GPIO permettent de commander ou lire des signaux numériques simples. Ils seront utilisés pour les LEDs, boutons et signaux de direction des moteurs.

UART

L'UART sert au debug et à la télémétrie. C'est un outil essentiel pour observer l'état interne du robot pendant les essais.

I2C

L'I2C permet de communiquer avec l'IMU. Il utilise deux lignes principales : SDA pour les données et SCL pour l'horloge.

PWM

Le PWM permet de contrôler la puissance moyenne envoyée aux moteurs. Il sera généré par les timers STM32.

Encodeurs

Les encodeurs mesurent le mouvement réel des moteurs. Avec deux voies A et B, ils permettent de connaître la vitesse, la position et le sens de rotation.

Filtre complémentaire

Le filtre complémentaire fusionne l'information rapide mais dérivative du gyroscope avec l'information plus stable mais bruitée de l'accéléromètre.

PID

Le PID corrige l'erreur entre une consigne et une mesure. Dans ce projet, il sera utilisé pour la vitesse des moteurs et pour l'équilibre du robot.

7. Architecture du robot

Architecture matérielle

Élément	Rôle
STM32 F446RE Nucleo-	Contrôle temps réel, lecture capteurs, PID, PWM
MPU6050	Mesure d'accélération et vitesse angulaire
TB6612FNG	Interface de puissance entre STM32 et moteurs
Moteurs + encodeurs	Action mécanique + mesure du mouvement
LiPo 2S	Source d'énergie principale
Buck LM2596	Conversion vers une tension logique stable
Châssis PVC expansé ex-	Structure mécanique du robot

Principe de fonctionnement

— L'IMU mesure l'inclinaison du robot.

- La STM32 estime l'angle avec un filtre complémentaire.
- Le PID calcule la correction nécessaire.
- La STM32 génère les signaux PWM.
- Le driver TB6612FNG applique la puissance aux moteurs.
- Les encodeurs mesurent le mouvement réel.

8. Les erreurs et leçons apprises

Erreur 1 – PID instable

Cause possible : gain proportionnel trop élevé.

Solution : réduire le gain proportionnel puis ajouter progressivement les autres termes.

Leçon : régler P avant I et D.

Erreur 2 – Robot qui tremble

Cause possible : bruit IMU, câbles moteurs trop proches des signaux, châssis flexible.

Solution : filtrage, séparation des câbles puissance/signaux, fixation rigide de l'IMU.

Leçon : un problème de stabilité n'est pas toujours un problème de code.

Erreur 3 – Reset de la STM32

Cause possible : chute de tension lors de l'appel de courant des moteurs.

Solution : alimentation logique séparée via Buck Converter et masse commune propre.

Leçon : l'alimentation est une partie critique du système.

9. Améliorations futures

- ESP32 pour Wi-Fi et télémétrie.
- Interface web pour visualiser les données en temps réel.
- Réglage PID à distance.
- Écran OLED pour afficher l'état du robot.
- Capteurs de distance VL53L0X.
- Caméra.
- Navigation autonome.
- SLAM simple.
- Passage à un châssis imprimé en 3D ou aluminium.

10. Matériel et plan d'achat

Matériel acheté pour la version 1

Composant	Rôle
STM32 Nucleo-F446RE	Carte principale temps réel
MPU6050	Mesure inertielle
TB6612FNG	Driver moteur
Deux motoréducteurs avec encodeurs	Actionnement et mesure de mouvement
LiPo 2S 7,4 V – 2200 mAh	Batterie principale
Chargeur LiPo avec équilibrage	Recharge sécurisée
Buck LM2596	Conversion tension logique
Connecteurs Deans + câbles silicone	Distribution puissance
Interrupteur ON/OFF	Coupure alimentation
Multimètre	Diagnostic électrique
Cutter	Fabrication châssis
PVC expansé	Structure mécanique
Fils Dupont et câbles	Connexions logiques et puissance

Budget

Le coût total de la version initiale du projet est estimé à :

200 €

Achats reportés

- ESP32 pour Wi-Fi ;
- écran OLED ;
- capteurs de distance ;
- caméra ;
- châssis avancé.

Conclusion

**Pourvu qu'aujourd'hui soit mieux
qu'hier**

Je ne cherche pas à être parfait.

Je cherche simplement à progresser.

Chaque ligne de code écrite.

Chaque bug corrigé.

Chaque composant compris.

Chaque test réalisé.

Me rapproche un peu plus de l'ingénieur que je veux devenir.

Le robot n'est pas la destination.

Le véritable projet, c'est moi.